Zofia Kapcińska 160227

Sprawozdanie: Implementacja algorytmu Diffiego-Hellmana uzgadniania klucza sesyjnego

1. Screenshot.

```python
import hashlib
import time
import random
import string
import numpy as np

def generateHash(text, algorithm):  4 usages
    hashObj = hashlib.new(algorithm)
    hashObj.update(text)
    return hashObj.hexdigest()

def findCollision(algorithm='sha256'):  1 usage
    hexNum = 12 // 4
    seen = {}
    attemps = 1

    while 1:
        randomText = ''.join(random.choices(string.ascii_letters + string.digits, k=10))
        fullHash = generateHash(randomText.encode('utf-8'), algorithm)
        prefix = fullHash[:hexNum]

        if prefix in seen:
            originalText = seen[prefix]
            originalHash = generateHash(originalText.encode('utf-8'), algorithm)
            print("\nZnaleziono kolizję po " + str(attemps) + " próbach:")
            print("Tekst 1: " + originalText + " | Hash: " + originalHash)
            print("Tekst 2: " + randomText + " | Hash: " + fullHash)
            print("Wspólny prefix: " + str(prefix))
            break

        seen[prefix] = randomText

def toBits(data):  1 usage
    bits = np.unpackbits(np.frombuffer(data, dtype=np.uint8))
    return bits

def sha256Bits(data):  2 usages
    hashObj = hashlib.new('sha256')
    hashObj.update(data)
    return toBits(hashObj.digest())
```

```python
def testSAC_sha256():  1 usage
    outputBytesLength = 256
    changeCounter = np.zeros(outputBytesLength)

    for _ in range(1000):
        inputBytes = np.random.bytes(32)
        hashBits = sha256Bits(inputBytes)

        modifiedInput = bytearray(inputBytes)

        byteID = random.randint( a: 0,  b: 31)
        bitID = random.randint( a: 0,  b: 7)

        modifiedInput[byteID] ^= (1 << bitID)
        modifiedHash = sha256Bits(modifiedInput)
        changeCounter += (hashBits ^ modifiedHash)

    return changeCounter / 1000
```

```python
if __name__ == '__main__':
    algorithms = ['md5', 'sha1', 'sha224', 'sha256', 'sha384', 'sha512', 'sha3_224', 'sha3_256', 'sha3_384', 'sha3_512']
    userText = input('Podaj tekst: ')

    testData = ["Lorem ipsum dolor sit amet, consectetur efficitur.",
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras nec sollicitudin arcu. Curabitur sed sodales nunc, vel feugiat lectus. Nullam tristique scelerisque libero eu ultrices. Suspendisse proin.
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque consequat mauris sed mi mollis accumsan. Aenean fermentum commodo risus a venenatis. Duis ut elit urna. Praesent in malesuada nisl, et
                "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed tempus gravida dolor, vitae molestie tellus tempus et. Aliquam malesuada posuere enim viverra finibus. Duis elementum felis libero, ut dict

    for algorithm in algorithms:
        start = time.time()
        val = generateHash(userText.encode('utf-8'), algorithm)
        end = time.time()
        print("\nAlgorytm " + algorithm + ": \nTekst użytkownika o długości " + str(len(userText)) + " znaków: " + val + " | Czas działania: " + str(end - start) + " | Długość ciągu wyjściowego: " + str(len(val)))
        for data in testData:
            start = time.time()
            val = generateHash(data.encode('utf-8'), algorithm)
            end = time.time()
            print("Tekst testowy o długości " + str(len(data)) + " znaków: " + val + " | Czas działania: " + str(end - start) + " | Długość ciągu wyjściowego: " + str(len(val)))

    findCollision('sha256')
    prob = testSAC_sha256()
    meanProb = np.mean(prob)
    print("\nŚrednie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: " + str(round(meanProb, 3)))
```

```
Podaj tekst: test1234

Algorytm md5:
Tekst użytkownika o długości 8 znaków: 1dd7a4fca7442dda3ad93c9a726597e4 | Czas działania: 5.1975250244140625e-05 | Długość ciągu wyjściowego: 32
Tekst testowy o długości 50 znaków: 13a82315419feb4d520d0baed43fc52d | Czas działania: 7.152557373046875e-06 | Długość ciągu wyjściowego: 32
Tekst testowy o długości 200 znaków: 252e74cec910f74d8d2a08902b571eac | Czas działania: 3.3378610742187 5e-06 | Długość ciągu wyjściowego: 32
Tekst testowy o długości 500 znaków: 2c5ebe759b45faa1cca4881a81a775a9 | Czas działania: 3.3378610742187 5e-06 | Długość ciągu wyjściowego: 32
Tekst testowy o długości 1000 znaków: 3e7c939a61f581b4bd3bc8441338039d | Czas działania: 3.5762786865234375e-06 | Długość ciągu wyjściowego: 32

Algorytm sha1:
Tekst użytkownika o długości 8 znaków: 9bc34549d565d9505b287de0cd20ac77be1d3f2c | Czas działania: 1.4543533325195312e-05 | Długość ciągu wyjściowego: 40
Tekst testowy o długości 50 znaków: 2e19fa048ffcd5ba5a9ea1eeb415035fbe32c4d4 | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 40
Tekst testowy o długości 200 znaków: f9db45c37a7e2f1d0cf3cea262835ae6545e18f1 | Czas działania: 1.6689300537109375e-06 | Długość ciągu wyjściowego: 40
Tekst testowy o długości 500 znaków: 72c9077cff114bffc65876ff09bfd8dd51d6c1bc | Czas działania: 2.1457672119140625e-06 | Długość ciągu wyjściowego: 40
Tekst testowy o długości 1000 znaków: a1c3f0aa3a1ebf776b1e27a423355c7c4b6773d | Czas działania: 2.1457672119140625e-06 | Długość ciągu wyjściowego: 40

Algorytm sha224:
Tekst użytkownika o długości 8 znaków: fe53be35763bccd7295b7225ed41f5c508440e8c4cc2a6b52725105a | Czas działania: 1.0013580322265625e-05 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 50 znaków: bccaeca8df2dd6e5bc556435ea638278a1a30d389fb0f6ad6165668d | Czas działania: 3.5762786865234375e-06 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 200 znaków: 4368987c92ee28d6b733fee387d21a850dee4d3b5b8f01758f273688 | Czas działania: 3.5762786865234375e-06 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 500 znaków: 706007ddecbff1f760101fc2322c102723d313ca783c43d128d38c8 | Czas działania: 4.2915344238281 25e-06 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 1000 znaków: 2cf9b8b43be9283db6848c9ac188e1dd529c4ebd94aaacfdcd9addfa | Czas działania: 4.2915344238281 25e-06 | Długość ciągu wyjściowego: 56

Algorytm sha256:
Tekst użytkownika o długości 8 znaków: 937e8d5fbb48bd4949536cd65b8d35c426b80d2f830c5c308e2cdec422ae2244 | Czas działania: 1.1205673217773438e-05 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 50 znaków: 2969696989bc6e7b6a54fe8d6087 2bb73020ba227e3e4b49e1568679a503e048 | Czas działania: 5.0067901611328125e-06 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 200 znaków: e608829a72950 7dc2141d255c7a75384ee4e5d8bee71c5bbf42827dccc53cc0b5 | Czas działania: 4.7683715820312 5e-06 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 500 znaków: 6c26b666101e859417fb75dcb36c93a39a4ec80c775f91ceb87973b46c19c2cc | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 1000 znaków: c68ffedda84e5a5f8e376f8df6a8cf85e188ef29e021128e5d0162dbe24a6c6c | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 64

Algorytm sha384:
Tekst użytkownika o długości 8 znaków: 25421d7c7e6fd6aa71f707353cc3408fa1fb316e698136553b0e03eea875a7808b70e1683004822283 07a09504d7fdd4 | Czas działania: 1.6450881958007812e-05 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 50 znaków: e9b1c2137b237f8625b6abdf8f43e8e62165bd8b457d7d71c0fe9fb2cbffaD77dd1c6185a63f5ef122a158412e8335f17 | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 200 znaków: 148e20ba8986a5eee042b38bcffd459459193a40a858b69e63e23b239ae17bac57db96ca92dccfd53e1ce71ea3291015 | Czas działania: 2.1457672119140625e-06 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 500 znaków: fd9c50a98efc4939b4f2372f0bbd36c080b95140aac3e126897544253a786a842ea19dc03e7218a c07ba452d6cb2378 | Czas działania: 8.5830088476562 5e-06 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 1000 znaków: 98323e4240bb518d138f36f9e9240c55a0fdaf1a01acfb3e2a468a449c72d1f44b49d6196b3ae18c3c0bd4e3bb130fba | Czas działania: 3.8146972656625e-06 | Długość ciągu wyjściowego: 96

Algorytm sha512:
Tekst użytkownika o długości 8 znaków: 2bbe0c48b91a7d1b8a6753a8b9cbe1db16b84379f3f91fe115621284df7a48f1cd71e9beb90ea614c7bd924250aa9e446a866725e685a65df5d139a5cd180dc9 | Czas działania: 8.1062318945312 5e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 50 znaków: 105485971131e51bc1c31828ed7745a148a81a5ac2974aa8e1045318a27df3dda62f25d0d5267643a169c2ade1fcd17a2d8a5665ba248c18e06e9dd5a67be757 | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 200 znaków: 910b78339b03e2dc88b7875504dbff4a123d443d7147dd262610043a9385cc2787899f4bf9d6679c1e79161e13eb03c22233b2b0cc15b6f2348ebbc32bf4ec282 | Czas działania: 1.6689300537109375e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 500 znaków: 5191548fa030 0ee76f2f2431202fa60fe59eb075105567 6fa46ae8c010c74852741e7684bf857483232721 3b88f3b8f1534bf56b311a3008d84ea293e17f1b2c | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 1000 znaków: d54243b528380cfd26df11c3f05002c6f3f71687c789e9b4df01c3b65211d5242579041e48505ff1eb46f9c75b1f5f50bc0ba75121cf55fd42bdc2f10d06ab37 | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 128
```

```
Algorytm sha3_224:
Tekst użytkownika o długości 8 znaków: 6f27da5c1d035d528c5517fb60431edc8b0cb343692da0c1977c7435 | Czas działania: 1.6212463378906 25e-05 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 50 znaków: 7cfb80c81928454fd7f6a25de98833b63334b3c2eee640 6cf97c55bb | Czas działania: 2.8610229492187 5e-06 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 200 znaków: cb8f76f46bd8a9ecfdde6ab952a4f3a606e671100890819 5a95e8d11 | Czas działania: 1.2874603271484375e-05 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 500 znaków: 93991c4bb93e81984989198 6b87fde072ffc82eeda83a03d7490f920 | Czas działania: 2.6226043701171875e-06 | Długość ciągu wyjściowego: 56
Tekst testowy o długości 1000 znaków: 1dbcd3f10a4d643751ce1511099f31225bc566cdfecb8a2169cf373f | Czas działania: 3.0994415283203125e-06 | Długość ciągu wyjściowego: 56

Algorytm sha3_256:
Tekst użytkownika o długości 8 znaków: 0a8c939670358327a6bec55af0f44041dea04a3fffff2f48d0e165a969b8b9e7 | Czas działania: 6.1988830566406 25e-06 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 50 znaków: f90a112ce25a16dd0f878fa9c3bfe56ad2e549d5188809ab97461f51a412f31b | Czas działania: 2.1457672119140625e-06 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 200 znaków: 513d5e959042d7220aa8e96fef3ad15a056e785f3689a7dd34363e1226aa5c30 | Czas działania: 1.9073486328125e-06 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 500 znaków: c8a8c91f1d12bcf92a21fc1bd330427558077da27c606ed9ee928130e5ecdaa44 | Czas działania: 2.3841857910156 25e-06 | Długość ciągu wyjściowego: 64
Tekst testowy o długości 1000 znaków: 756f210825025154 4b98c923ca09149c5ffaee79c89aafbd9a94127d4005 8f62 | Czas działania: 3.3378610742187 5e-06 | Długość ciągu wyjściowego: 64

Algorytm sha3_384:
Tekst użytkownika o długości 8 znaków: 831b07bde03dfc17a0a9bee562447eec7155c2c2ba9e71c2e5f7aed870540253af707b382862ea2128ffabb6fadf5e5e | Czas działania: 4.2915344238281 25e-06 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 50 znaków: 2d8d169a2bb4c4398fe9b39e067b989b9d237f26a2f0108f6cdbd4e988866a954fa8ec7a5481bd7270b8fcc4e50924e0 | Czas działania: 1.6689300537109375e-06 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 200 znaków: 8ffaae8a59e42ea17534c899ba9bbd98f427941f89f6582fe2149c727f4a44c1128d15a598a1e10bd6c95a0413f1ec03 | Czas działania: 1.6689300537109375e-06 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 500 znaków: 9b10493e3717416ac0f3fdfbdda9c957f6d092272e1f602498816 8b6ca31f00c010a4e1aa101bb8c4384db773ee01b68 | Czas działania: 2.6226043701171875e-06 | Długość ciągu wyjściowego: 96
Tekst testowy o długości 1000 znaków: 988dc5b789683de5b03 6278f8deb14af9685ca8d9e012c2342909cb792cd1345698 8b5fa84a07fb483c1226c1bcc7d54 | Czas działania: 3.5762786865234375e-06 | Długość ciągu wyjściowego: 96

Algorytm sha3_512:
Tekst użytkownika o długości 8 znaków: c9a9d2e15311717e61ef5a91e0800271a619edaed0fe0eada11e68e3c316bef64be59ae9b1a9c9cfaaf1851ae71b20746d8f8599bca29931cb5e6983b7029d9 | Czas działania: 4.0531158447265625e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 50 znaków: 7001be61727290b2afbd1771f0e53256601239 5b517f561a3a5f7c6b7656d96223dd94f831b52ea0a571aa2f3e90d0e31ba15bafed9b69112f0a500faecdaf42 | Czas działania: 1.4305114746009375e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 200 znaków: d95e21f2c505222f08833f8fcd0d6a7687af76c9b25a18f1152632ff1263 0ed2be1e520f6e2468a679dcd8f70deb6a2a47cf9bc3f0100a568 6c9f6c4caf4ae64 | Czas działania: 1.6689300537109375e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 500 znaków: ddc7f3223f3b4bae165053e7366224692a96a725f92271df76daf133d97c155dc675d3b68798c341a57a98167cb7a16d0863dd f09e2327129a5bb27b90088366 | Czas działania: 2.8610229492187 5e-06 | Długość ciągu wyjściowego: 128
Tekst testowy o długości 1000 znaków: 3f90e8c0f531b400e84de56bb59da54ad46d3a1575eb7fbd41589e4c09226f32c35ac9a3b5654baddd47afe3fd7f28ea7f295271dfe01dcf7db2f5be3c1143b | Czas działania: 4.2915344238281 25e-06 | Długość ciągu wyjściowego: 128

Znaleziono kolizję po 1 próbach:
Tekst 1: wX6PXXcHyq | Hash: 2cc4df70640d96dab20873b6d072af3b2a283c6b20773dfcb5c33e4094ca8652
Tekst 2: UhlMpItcpA | Hash: 2ccd78576d3bcd9174156e34f76ecd4fdc04c5dff20a65639bdd1af66924574c
Wspólny prefix: 2cc

Średnie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: 0.5
```

2. **Omówienie sposobu implementacji.**
   Aplikacja została zaimplementowana w języku Python z użyciem bibliotek:
   - hashlib, zawierającej natywny dostęp do algorytmów kryptograficznych
   - time, pozwalającej na zmierzenie czasu działania algorytmów
   - random, pozwalającej na wygenerowanie danych do testów oraz wybranie losowych indeksów
   - string
   - numpy

   Do generowania skrótów użyłam funkcji hashlib.new(), która pozwala na dynamiczny wybór algorytmów.

   W badaniu kolizji wykorzystałam słownik, w którym zapisywane są 12-bitowe prefiksy oraz oraz odpowiadający im losowo wygenerowany tekst. Sprawdzam, czy prefiksy występują wielokrotnie, co oznacza kolizję.

   Do zbadania własności SAC generowane jest 256 bitów oraz funkcja skrótu. Następnie zmieniany jest jeden losowy bit w oryginalnym ciągu (XOR) i ponownie generowany jest skrót. Badana jest zmiana bitów między skrótami z użyciem XOR-a oraz obliczone zostaje prawdopodobieństwo zmiany.

3. **Określenie roli soli w tworzeniu skrótów.**
   Sól to losowy ciąg danych dodawany do hasła przed procesem hashowania. Zapewnia to unikalną zmianę wyniku, dzięki czemu nawet identyczne hasła mają różne skróty. Zwiększa to bezpieczeństwo i zapobiega łamaniu haseł.

4. **Na podstawie powszechnie dostępnych źródeł odpowiedz na pytanie – czy funkcję MD5 można uznać za bezpieczną? Czy dotychczas zostały znalezione dla niej jakiekolwiek kolizje?**
   Funkcja MD5 nie może być uznana za bezpieczną. Jest mało odporna na ataki oraz podatna na kolizje, które zostały znalezione także w moich testach.

5. **Zestawienie wyników wraz ze stosownymi wnioskami.**
   1) Porównanie algorytmów

Algorytm	**Długość na wyjściu (w bitach)**	**Czas działania (50 znaków)**	**Czas działania (200 znaków)**	**Czas działania (500 znaków)**	**Czas działania (1000 znaków)**
MD5	128	7.39 * 10^(-6)	3.1 * 10^(-6)	3.1 * 10^(-6)	3.58 * 10^(-6)
SHA-1	160	2.62 * 10^(-6)	1.67 * 10^(-6)	1.91 * 10^(-6)	2.15 * 10^(-6)
SHA-2 (224)	224	2.62 * 10^(-6)	3.1 * 10^(-6)	2.62 * 10^(-6)	3.34 * 10^(-6)
SHA-2 (256)	256	2.62 * 10^(-6)	2.15 * 10^(-6)	1.91 * 10^(-6)	2.38 * 10^(-6)
SHA-2 (384)	384	2.15 * 10^(-6)	2.38 * 10^(-6)	8.82 * 10^(-6)	3.57 * 10^(-6)
SHA-2 (512)	512	2.86 * 10^(-6)	1.67 * 10^(-6)	2.15 * 10^(-6)	2.62 * 10^(-6)
SHA-3 (224)	224	2.86 * 10^(-6)	2.62 * 10^(-6)	3.34 * 10^(-6)	4.05 * 10^(-6)
SHA-3 (256)	256	2.62 * 10^(-6)	2.62 * 10^(-6)	3.34 * 10^(-6)	4.77 * 10^(-6)
SHA-3 (384)	384	2.38 * 10^(-6)	2.62 * 10^(-6)	3.34 * 10^(-6)	4.77 * 10^(-6)
SHA-3 (512)	512	1.91 * 10^(-6)	2.38 * 10^(-6)	3.81 * 10^(-6)	5.72 * 10^(-6)

   2) Kolizje
      Znaleziono kolizję po 1 próbach:
      Tekst 1: 7G9tmmpGL8
      Hash:
      7be0e08890612812d111c2dda31947151ecb1e418042518a2c7feb478c5c0e7b

      Tekst 2: LF8HZgKRhS
      Hash:
      7be4971570be5267513e421dec7cdaf619f29eaba7f3df0c1266df22559657dd

      Wspólny prefix: 7be

   3) SAC
      Średnie prawdopodobieństwo zmiany dla wszystkich bitów na wyjściu po zmianie jednego bitu na wejściu: 0.5

Wnioski:

Niezależnie od długości tekstu wejściowego (od 50 do 1000 znaków), dany algorytm zawsze generuje skrót o identycznej liczbie znaków. Czas obliczeń rośnie bardzo powoli wraz ze wzrostem objętości danych, ale nie we wszystkich algorytmach (np. dla SHA-2 224). Dla 50 znaków większość algorytmów potrzebowała więcej czasu. Algorytmy z rodziny SHA-3 oraz wersje o dłuższych skrótach wymagają zazwyczaj nieco więcej operacji, co jest widoczne przy przetwarzaniu większych bloków danych. Im dłuższy ciąg wyjściowy, tym trudniejsze jest znalezienie kolizji, co bezpośrednio przekłada się na wyższy poziom bezpieczeństwa kryptograficznego